

Digital Image Processing

Extra Team Project Report

Team 15

50251*66 데니스 고레츠키

20206224 이주환



Contents

Outline of Program	2
Purpose	2
Functionality Overview.....	2
Design of Program	2
High-Level Design.....	2
Flow Diagram	3
Implementation Notes	3
How to execute.....	4
Time spent	5
Process of Project and Record of Project Meetings	5
Meeting 1 – Project Kickoff.....	5
Meeting 2 – Review & Integration	5
Records	5
GitHub records.....	6
Member's Thoughts on This Project	6
Lessons Learned	6
Future Improvements	6

Outline of Program

Purpose

Following the requirements for the extra project, the goal of this program is to determine whether the shape in a given image is a Circle or a Triangle. To achieve this, the program implements basic image processing algorithms and utilizes the geometric features of the object to perform the classification.

Functionality Overview

The program classifies the shape in an image through the following sequence of operations:

- **Image Loading:** Loads the input image using the OpenCV library.
- **Grayscale Conversion:** Converts the color image to grayscale to remove color information.
- **Binarization:** Separates the object (white) from the background (black) based on a specified threshold.
- **Contour Extraction:** Finds the coordinates of the object's contour from the binarized image.
- **Feature Extraction:** Calculates the Area and Perimeter from the extracted contour information, and uses these to derive the Circularity.
- **Shape Classification:** Based on the calculated circularity value, the program makes a final classification of whether the shape is a circle or a triangle.

Design of Program

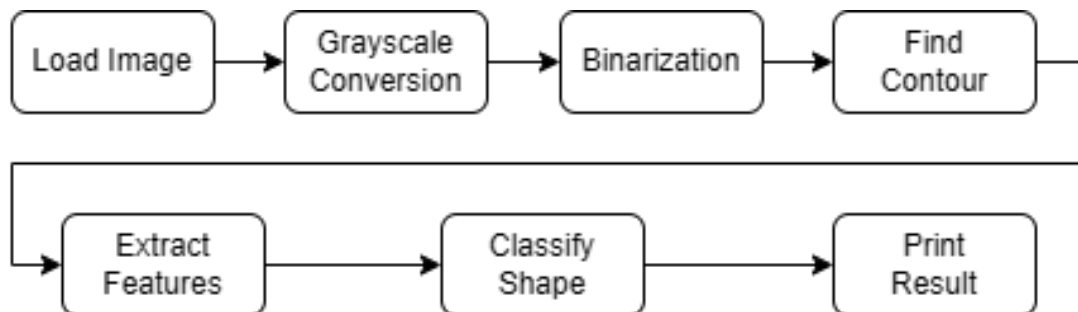
High-Level Design

The program is structured into three main modules based on functionality:

- **Image Processing Module (image.cpp, image.h):**
 - This module is responsible for the preprocessing pipeline, including image loading, grayscale conversion, binarization, and contour extraction.
 - Adhering to the project constraints, key object detection-related functions (binarization, contour extraction, etc.) were implemented from scratch without using their pre-built library equivalents.

- **Feature Extraction & Classification Module (feature.cpp, feature.h):**
 - Receives the contour data from the Image Processing Module and calculates key features such as area, perimeter, and circularity.
 - Contains the logic that uses these features to perform the final shape classification.
- **Main Module (main.cpp):**
 - Orchestrates the overall workflow by calling functions from the other modules in the correct sequence.
 - Prints the final classification result to the console.

Flow Diagram



Implementation Notes

- **Core Classification Logic:** The key to the classification is the Circularity metric. Circularity is calculated using the formula $(4 * \pi * \text{Area}) / (\text{Perimeter}^2)$. In theory, a perfect circle has a circularity value of 1.0. In this project, if the value is greater than a certain threshold (0.8), the shape is classified as a circle; otherwise, it is classified as a triangle.
- **Algorithm Details:**
 - **Image Loading (loadImage):** The implementation uses `cv::imdecode` to load the image by first reading the entire file into a buffer. This method ensures that images can be loaded reliably, even if the file path contains non-ASCII characters.
 - **Binarization (threshold):** This function inverts the image, setting the object to white (255) and the background to black (0). This step is crucial preparation for the contour extraction algorithm, which is designed to trace white pixels.
 - **Contour Extraction (findContour):** A custom contour-tracing algorithm was implemented. It operates by first scanning the image to find a starting pixel of the object. From there, it traces the object's boundary by

searching neighboring pixels in a clockwise order, storing the coordinates of the contour until it returns to the starting point.

- **Execution Structure:** The main function in main.cpp is structured with a for loop to sequentially process a total of four image files, from 1.jpg to 4.jpg. For each file, it performs the entire pipeline of image processing, feature extraction, and shape classification before printing the result to the console.

How to execute

To build and run this project, your environment must be configured correctly. The following steps detail the setup process in Visual Studio with the OpenCV library.

1. Configure Project Properties First, configure the project to link with the OpenCV library correctly. Open the project's **Properties** page and ensure the **Platform** is set to **x64**.

- **C/C++ > General > Additional Include Directories:** Add the path to your OpenCV's build\include folder. (*Example: C:\opencv\build\include*)
- **Linker > General > Additional Library Directories:** Add the path to the lib folder that matches your Visual Studio version. (*Example for VS 2019: C:\opencv\build\x64\vc16\lib*)
- **Linker > Input > Additional Dependencies:** Add the names of the OpenCV library files. (Replace XXXX with your version number).
 - opencv_worldXXXX.lib
 - opencv_worldXXXXd.lib

2. Prepare Execution Files

- **Copy DLL Files:** Find the opencv_worldXXXX.dll file in your OpenCV bin folder (e.g., C:\opencv\build\x64\vc16\bin) and copy it into the project's build output folder (e.g., YourProjectFolder\x64\Release).
- **Copy Image Files:** Place the input image files (1.jpg, 2.jpg, etc.) into the main project folder where the .vcxproj file is located.

3. Build and Run

- In the Visual Studio toolbar, set the solution configuration to **Release** and the platform to **x64**.
- Press **Ctrl+F5** or select **Debug > Start Without Debugging** to compile and run the program. The results will be printed to the console window.

Time spent

Planning & Design: ~1 hour

Core Algorithm Implementation (Contour Finding, Feature Calculation): ~3 hours

Visual Studio Environment Setup & Debugging: ~2 hours

Report Writing & Finalization: ~1 hour

Total Estimated Time: ~7 hours

Process of Project and Record of Project Meetings

This extra project was undertaken separately from the main team project. The initial logic was developed in a macOS environment. The main focus of the work then shifted to porting the code to a Windows environment to run in Visual Studio.

The most significant challenge was resolving issues related to linking the OpenCV library in Visual Studio. Persistent errors related to the inability to load image files arose from incorrect path settings, linker dependencies, and library incompatibilities between Debug and Release modes. After attempting various solutions, including absolute path testing and modifying runtime library settings, the program was finally made to run successfully in **Release mode**.

need to change below

Meeting 1 – Project Kickoff

- Discussed project requirements
- Assigned tasks: one team member focused on the composite function, the other one on the clamping
- Duration: ~30 minutes

Meeting 2 – Review & Integration

- Shared individual implementations
- Tested composite function with sample images
- Adjusted clamping logic to handle edge cases
- Duration: ~30 minutes

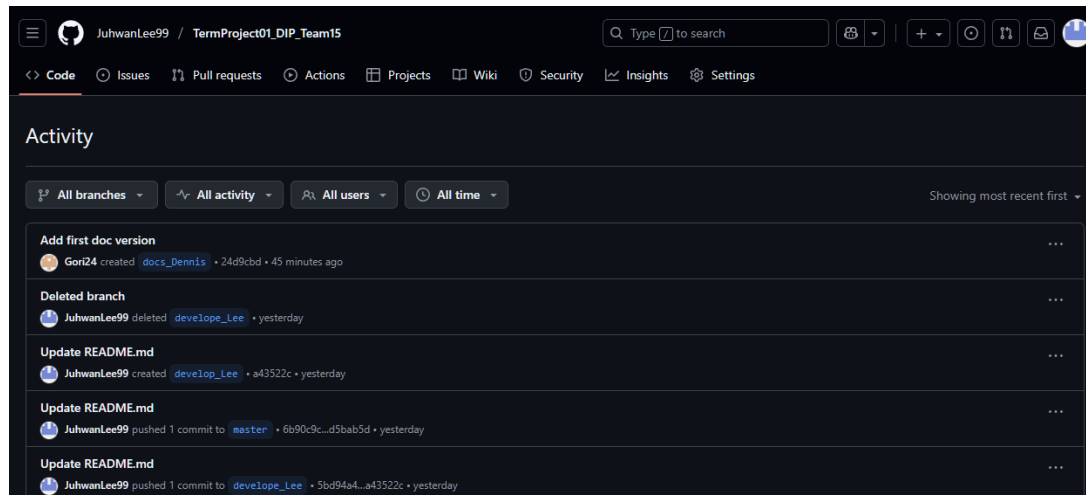
Records

- TODO: KakaoTalk screenshots
 - Example: discussion on how to handle overflow/underflow
1. Adjusting schedule

2. Discussing how to share the code

3. Next step

GitHub records



update the screenshot before submitting

Member's Thoughts on This Project

Lessons Learned

- **Importance of Algorithms:** Implementing image processing algorithms like contour extraction from scratch, rather than just using a library function, provided a deep understanding of their internal mechanics.
- **Understanding the Development Environment:** This project was an opportunity to experience the differences between operating systems and IDEs. It was a lesson in navigating the complexities of Visual Studio's project properties. The debugging process reinforced the importance of a systematic approach to problem-solving.
- **Application of Mathematical Principles:** It was insightful to see how a mathematical concept like circularity could be directly applied to classify objects in a real image, connecting theory with practical application.

Future Improvements

- **Support for More Shapes:** The program could be extended to classify more polygons, such as squares and rectangles, by adding additional feature extraction logic.

- **Improved Algorithm Robustness:** The current contour-finding algorithm works well for simple, clean images but may be less effective with noisy or complex images. The algorithm could be enhanced to improve its robustness.
- **Multiple Object Handling:** A future version could include functionality to detect and classify multiple objects within a single image, making the program more practical.